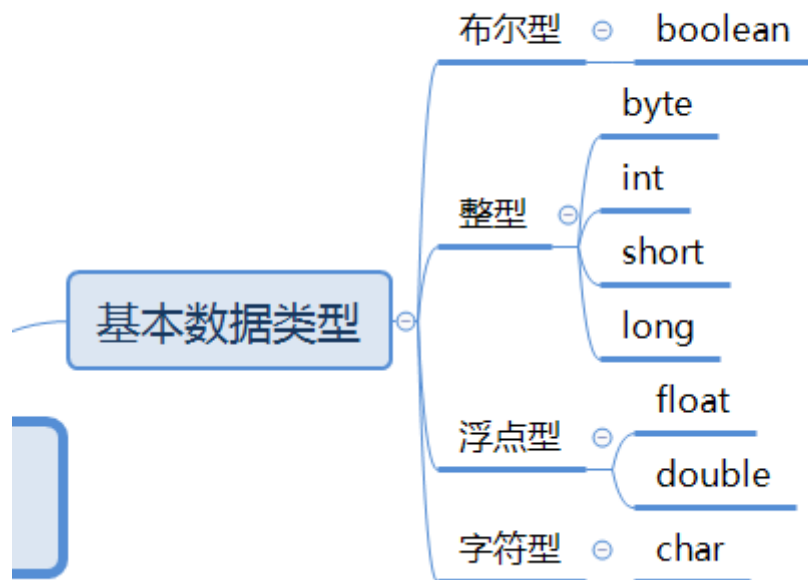


8大基础数据类型



1.逻辑型(或称布尔型)

所占内存: 占1个字节

常量: true , false

变量: 使用boolean定义:

```
boolean b;
```

```
boolean bo=true;
```

boolean 是否=true; (不建议变量名用中文)

2.byte(字节型)

所占内存: 占1个字节

常量: 12、32(十进制); 033(八进制); 0x7F (十六进制)

变量: 使用byte定义:

```
byte by1;
```

```
byte by=033;
```

```
byte 年龄=55;
```

3.short整型

所占内存: 占2个字节

常量: 12、32(十进制); 01133(八进制); 0x117F (十六进制)

变量: 使用short定义:

```
short sh1;
```

```
short sh2=12;
```

```
short 速度=180;
```

4.int 整型

所占内存: 占4个字节

常量: 12、32(十进制); 01133(八进制); 0x117F (十六进制)

变量: 使用int定义:

```
int in1;
```

```
int in2=12;  
int 速度=180;
```

5.long 整型

所占内存: 占8个字节

常量: 12、32(十进制); 01133(八进制); 0x117F (十六进制)

Java 里使用 `long` 类型的数据一定要在数值后面加上 `L`，否则将作为整型解析。

变量: 使用long定义:

```
long lo1;  
long lo2=185l;  
long 速度=180L;
```

6.float

所占内存: 占4个字节

常量: 3.14F、2.23f;

必须数字后要大写或小写的 `f`

变量: 使用float定义:

```
float fl1;  
float fl2=12f;  
float 收入=752.2F;
```

7.double

所占内存: 占8个字节

常量: 3.14D、2.23d、23.23;

数字后要大写或小写的 `d` (可选)

变量: 使用double定义:

```
double do1;  
double do2=18;  
double 收入=500;  
double 收入2=500d;
```

8.char型; java使用Unicode字符集;

所占内存: 占2个字节

字符型用单引号括起来

常量: 'a'、'?', '!'

变量: 使用char定义:

```
char ch1;  
char ch2='v';  
char 制表符='\t';
```

传参方式

值传递: 八种基本数据类型和String

值传递: 方法调用时, 实际参数把它的值传递给对应的形式参数, 方法执行中形式参数值的改变不影响实际参数的值

引用传递: 除String以外的所有复合数据类型, 包括数组、类和接口

引用传递: 也称为传地址。方法调用时, 实际参数的引用(地址, 而不是参数的值)被传递给方法中相对应的形式参数, 在方法执行中, 对形式参数的操作实际上就是对实际参数的操作, 方法执行中形式参数值的改变将会影响实际参数的值

jdk8的新特性

- Lambda表达式
- 函数式接口
- *方法引用和构造器调用
- Stream API
- 接口中的默认方法和静态方法
- 新时间日期API

反射创建对象

动态的获取信息以及动态调用对象的方法: 3种方式

- 1 通过Object类的getClass方法来获取
- 2 使用类名加“.class”的方式即会返回与该类对应的Class对象。
- 3 使用Class.forName方法

成员变量与局部变量的区别？

- **语法形式**：从语法形式上看，成员变量是属于类的，而局部变量是在代码块或方法中定义的变量或是方法的参数；成员变量可以被 `public`, `private`, `static` 等修饰符所修饰，而局部变量不能被访问控制修饰符及 `static` 所修饰；但是，成员变量和局部变量都能被 `final` 所修饰。
- **存储方式**：从变量在内存中的存储方式来看，如果成员变量是使用 `static` 修饰的，那么这个成员变量是属于类的，如果没有使用 `static` 修饰，这个成员变量是属于实例的。而对象存在于堆内存，局部变量则存在于栈内存。
- **生存时间**：从变量在内存中的生存时间上看，成员变量是对象的一部分，它随着对象的创建而存在，而局部变量随着方法的调用而自动生成，随着方法的调用结束而消亡。
- **默认值**：从变量是否有默认值来看，成员变量如果没有被赋初始值，则会自动以类型的默认值而赋值（一种情况例外：被 `final` 修饰的成员变量也必须显式地赋值），而局部变量则不会自动赋值。

数组和链表的区别

- 数组支持随机访问，而链表不支持。
- 数组使用的是连续内存空间对 CPU 的缓存机制友好，链表则相反。
- 数据的大小固定，而链表则天然支持动态扩容。

hashmap

在JDK1.7和1.8的实现不同；1.7是数组+链表；1.8是数组+链表+红黑树

1.7

根据key计算index，算出hash值后， $index = (n-1) \& hash$ 计算index，将键值对插入数组中下标为index的位置；在插入新元素时，如果计算出的index与之前的相同，发生hash冲突，使用头插法用链表的形式插入到index=1的位置，在JDK1.8后改成尾插法。如果index不同则插入到别的index位置。

1.8

每发生一次hash冲突，链表长度都会+1，当链表长度>8时，先调用treeifybin方法，会根据数组的长度来决定是否将链表转变为红黑树，当数组长度>=64的情况下会将链表转为红黑树，如果<64则进行扩容

hashmap扩容

capacity：默认容量是16

loadFactor：负载因子，默认0.75f

扩容后是原来容量的2倍

B+树的特点和优势

关键字数量跟路数相等；

根节点和枝节点上都不会存储数据，只在叶子节点存储数据；

每个叶子节点增加了一个指向相邻叶子节点的指针，形成一个有序链表的结构。

优势

1)它是 B Tree 的变种，B Tree 能解决的问题，它都能解决。B Tree 解决的两大问题是（每个节点存储更多关键字；路数更多）

2)扫库、扫表能力更强（如果我们要对表进行全表扫描，只需要遍历叶子节点就可以了，不需要遍历整棵 B+Tree 拿到所有的数据）

3) B+Tree 的磁盘读写能力相对于 B Tree 来说更强，同数据量下磁盘I/O次数更少（根节点和枝节点不保存数据区，所以一个节点可以保存更多的关键字，一次磁盘加载的关键字更多）

4)范围查询和排序能力更强（因为叶子节点上有下一个数据区的指针，数据形成了链表）

5)效率更加稳定（B+Tree 永远是在叶子节点拿到数据，所以 IO 次数是稳定的）

优先队列

保证每次从队列中取出的元素都是权值最小的

通过完全二叉树实现的小顶堆（任意一个非叶子节点的权值，都不大于其左右子节点的权值）

红黑树和最小堆

红黑树插入最差要比较 $2\log N$ ，最小堆最坏情况是 $\log N$

红黑树删除不需要比较，只需要不超过3次旋转，而最小堆需要 $2\log N$ 次

最小堆最大的优势在于取最小值性能是 $O(1)$ ，如果插入的数据基本有序，那么最小堆能获得比较好的性能，在频繁不断地取最小值的是否满足条件的应用中有更加好的性能（如超时队列），红黑树相应就要尽量避免不必要的查询次数才能在超时队列这种应用中获得更好的性能

面向对象的理解

JAVA将对象视为一种奇特的变量，它除了可以存储数据之外还可以对它自身进行操作。

它能够直接反映现实生活中的事物，例如人、车、小鸟等，将其表示为程序中的对象。

每个对象都具有各自的状态特征（也可以称为属性）及行为特征（方法），java就是通过对象之间行为的交互来解决问题的。

面向对象和面向过程的区别

面向过程

优点：性能比面向对象好，因为类调用时需要实例化，开销比较大，比较消耗资源。

缺点：不易维护、不易复用、不易扩展

面向对象

优点：易维护、易复用、易扩展，由于面向对象有封装、继承、多态性的特性，可以设计出低耦合的系统，使系统更加灵活、更加易于维护

缺点：性能比面向过程差

面向对象的三大特性

类具有三个特性：封装、继承和多态。

1、封装(Encapsulation)

封装是指属性私有化

- 根据需要提供setter和getter方法来访问属性
- 隐藏具体属性和实现细节，仅对外开放接口
- 控制程序中属性的访问级别

目的：增强数据安全性，不能让其他用户随意访问和修改数据，简化编程，使用者不必在意具体实现细节，而只是通过外部接口即可访问类的成员

2、继承 (Extend)

继承是指将多个相同的属性和方法提取出来，新建一个父类

- java中一个类只能继承一个父类，且只能继承访问权限非private属性和方法
- 子类可以重写父类中的方法，命名与父类中同名的属性

目的：代码复用

3、多态

多态 (Polymiorph) 可以分为两种：设计时多态、运行时多态

- 设计（编译）时多态：即重载（Overload），是指java允许方法名相同而参数不同（返回值可以相同也可以不同），同一个类中允许存在一个或多个以上的同名函数，只要参数类型或参数个数不同即可
- 运行时多态：即重写（Override）必须是在继承体系中，子类重写父类方法，JVM运行时根据调用该方法的类型决定调用那个方法

目的：增加代码的灵活度

重载和重写有什么区别？

重载就是同样的一个方法能够根据输入数据的不同，做出不同的处理

重写就是当子类继承自父类的相同方法，输入数据一样，但要做出有别于父类的响应时，你就要覆盖父类方法

重载

发生在同一个类中（或者父类和子类之间），方法名必须相同，参数类型不同、个数不同、顺序不同，方法返回值和访问修饰符可以不同。

《Java 核心技术》这本书是这样介绍重载的：

如果多个方法(比如 `StringBuilder` 的构造方法)有相同的名字、不同的参数，便产生了重载。

```
1 | StringBuilder sb = new StringBuilder();  
2 | StringBuilder sb2 = new StringBuilder("HelloWorld");
```

1
2

编译器必须挑选出具体执行哪个方法，它通过用各个方法给出的参数类型与特定方法调用所使用的值类型进行匹配来挑选出相应的方法。如果编译器找不到匹配的参数，就会产生编译时错误，因为根本不存在匹配，或者没有一个比其他的更好(这个过程被称为重载解析(overloading resolution))。

Java 允许重载任何方法，而不只是构造器方法。

综上：重载就是同一个类中多个同名方法根据不同的传参来执行不同的逻辑处理。

重写

重写发生在运行期，是子类对父类的允许访问的方法的实现过程进行重新编写。

1. 方法名、参数列表必须相同，子类方法返回值类型应比父类方法返回值类型更小或相等，抛出的异常范围小于等于父类，访问修饰符范围大于等于父类。
2. 如果父类方法访问修饰符为 `private/final/static` 则子类就不能重写该方法，但是被 `static` 修饰的方法能够被再次声明。
3. 构造方法无法被重写

综上：重写就是子类对父类方法的重新改造，外部样子不能改变，内部逻辑可以改变。

方法的重写要遵循“两同两小一大”（以下内容摘录自《疯狂 Java 讲义》，[issue#892open in new window](#)）：

- “两同”即方法名相同、形参列表相同；
- “两小”指的是子类方法返回值类型应比父类方法返回值类型更小或相等，子类方法声明抛出的异常类应比父类方法声明抛出的异常类更小或相等；
- “一大”指的是子类方法的访问权限应比父类方法的访问权限更大或相等。

☆ 关于 **重写的返回值类型** 这里需要额外多说明一下，上面的表述不太清晰准确：如果方法的返回类型是 `void` 和基本数据类型，则返回值重写时不可修改。但是如果方法的返回值是引用类型，重写时是可以返回该引用类型的子类的。

深拷贝和浅拷贝区别了解吗？什么是引用拷贝？

关于深拷贝和浅拷贝区别，我这里先给结论：

- **浅拷贝**：浅拷贝会在堆上创建一个新的对象（区别于引用拷贝的一点），不过，如果原对象内部的属性是引用类型的话，浅拷贝会直接复制内部对象的引用地址，也就是说拷贝对象和原对象共用同一个内部对象。
- **深拷贝**：深拷贝会完全复制整个对象，包括这个对象所包含的内部对象。

上面的结论没有完全理解的话也没关系，我们来看一个具体的案例！

浅拷贝

浅拷贝的示例代码如下，我们这里实现了 `Cloneable` 接口，并重写了 `clone()` 方法。

`clone()` 方法的实现很简单，直接调用的是父类 `Object` 的 `clone()` 方法。

```
1 public class Address implements Cloneable{
2     private String name;
3     // 省略构造函数、Getter&Setter方法
4     @Override
5     public Address clone() {
6         try {
7             return (Address) super.clone();
```

```

8         } catch (CloneNotSupportedException e) {
9             throw new AssertionError();
10        }
11    }
12 }
13
14 public class Person implements Cloneable {
15     private Address address;
16     // 省略构造函数、Getter&Setter方法
17     @Override
18     public Person clone() {
19         try {
20             Person person = (Person) super.clone();
21             return person;
22         } catch (CloneNotSupportedException e) {
23             throw new AssertionError();
24         }
25     }
26 }

```

测试：

```

1 Person person1 = new Person(new Address("武汉"));
2 Person person1Copy = person1.clone();
3 // true
4 System.out.println(person1.getAddress() == person1Copy.getAddress());

```

从输出结构就可以看出，`person1` 的克隆对象和 `person1` 使用的仍然是同一个 `Address` 对象。

深拷贝

这里我们简单对 `Person` 类的 `clone()` 方法进行修改，连带着要把 `Person` 对象内部的 `Address` 对象一起复制。

```

1 @Override
2 public Person clone() {
3     try {
4         Person person = (Person) super.clone();
5         person.setAddress(person.getAddress().clone());
6         return person;
7     } catch (CloneNotSupportedException e) {
8         throw new AssertionError();
9     }
10 }

```

测试：

```

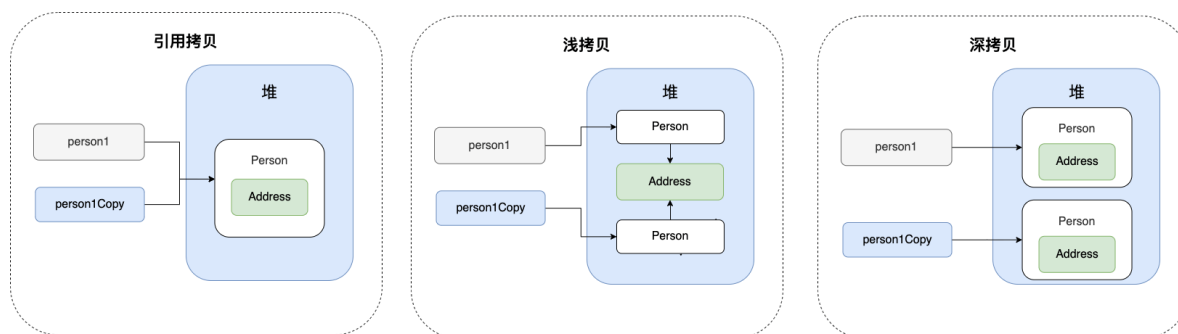
1 Person person1 = new Person(new Address("武汉"));
2 Person person1Copy = person1.clone();
3 // false
4 System.out.println(person1.getAddress() == person1Copy.getAddress());

```

从输出结构就可以看出，虽然 `person1` 的克隆对象和 `person1` 包含的 `Address` 对象已经是不同的了。

那什么是引用拷贝呢？ 简单来说，引用拷贝就是两个不同的引用指向同一个对象。

我专门画了一张图来描述浅拷贝、深拷贝、引用拷贝：



五大基本原则

1. 单一职责原则SRP(Single Responsibility Principle)

类的功能要单一，不能包罗万象，跟杂货铺似的。

2. 开放封闭原则OCP(Open - Close Principle)

一个模块对于拓展是开放的，对于修改是封闭的，想要增加功能热烈欢迎，想要修改，哼，一万个不乐意。

3. 里式替换原则LSP(the Liskov Substitution Principle LSP)

子类可以替换父类出现在父类能够出现的任何地方。比如你能代表你爸去你姥姥家干活。

4. 依赖倒置原则DIP(the Dependency Inversion Principle DIP)

高层次的模块不应该依赖于低层次的模块，他们都应该依赖于抽象。抽象不应该依赖于具体实现，具体实现应该依赖于抽象。就是你出国要说你是中国人，而不能说你是哪个村子的。比如说中国人是抽象的，下面有具体的xx省，xx市，xx县。你要依赖的抽象是中国人，而不是你是xx村的。

5. 接口分离原则ISP(the Interface Segregation Principle ISP)

设计时采用多个与特定客户类有关的接口比采用一个通用的接口要好。就比如一个手机拥有打电话，看视频，玩游戏等功能，把这几个功能拆分成不同的接口，比在一个接口里要好的多。

== 和 equals() 的区别

== 对于基本类型和引用类型的作用效果是不同的：

- 对于基本数据类型来说，**==** 比较的是值。
- 对于引用数据类型来说，**==** 比较的是对象的内存地址。

因为 Java 只有值传递，所以，对于 **==** 来说，不管是比较基本数据类型，还是引用数据类型的变量，其本质比较的都是值，只是引用类型变量存的值是对象的地址。

`equals()` 不能用于判断基本数据类型的变量，只能用来判断两个对象是否相等。`equals()` 方法存在于 `Object` 类中，而 `Object` 类是所有类的直接或间接父类，因此所有的类都有 `equals()` 方法。

`Object` 类 `equals()` 方法：


```
1 public boolean equals(Object obj) {
2     return (this == obj);
3 }
```

`equals()` 方法存在两种使用情况：

- **类没有重写 `equals()` 方法**：通过 `equals()` 比较该类的两个对象时，等价于通过“`==`”比较这两个对象，使用的默认是 `Object` 类 `equals()` 方法。
- **类重写了 `equals()` 方法**：一般我们都重写 `equals()` 方法来比较两个对象中的属性是否相等；若它们的属性相等，则返回 `true` (即，认为这两个对象相等)。

举个例子 (这里只是为了举例。实际上，你按照下面这种写法的话，像 IDEA 这种比较智能的 IDE 都会提示你将 `==` 换成 `equals()`)：

```
1 String a = new String("ab"); // a 为一个引用
2 String b = new String("ab"); // b为另一个引用,对象的内容一样
3 String aa = "ab"; // 放在常量池中
4 String bb = "ab"; // 从常量池中查找
5 System.out.println(aa == bb); // true
6 System.out.println(a == b); // false
7 System.out.println(a.equals(b)); // true
8 System.out.println(42 == 42.0); // true
```

`String` 中的 `equals` 方法是被重写过的，因为 `Object` 的 `equals` 方法是比较的对象的内存地址，而 `String` 的 `equals` 方法比较的是对象的值。

当创建 `String` 类型的对象时，虚拟机会在常量池中查找有没有已经存在的值和要创建的值相同的对象，如果有就把它赋给当前引用。如果没有就在常量池中重新创建一个 `String` 对象。

`String` 类 `equals()` 方法：

```
1 public boolean equals(Object anObject) {
2     if (this == anObject) {
3         return true;
4     }
5     if (anObject instanceof String) {
6         String anotherString = (String)anObject;
7         int n = value.length;
8         if (n == anotherString.value.length) {
9             char v1[] = value;
10            char v2[] = anotherString.value;
11            int i = 0;
12            while (n-- != 0) {
13                if (v1[i] != v2[i])
14                    return false;
15                i++;
16            }
17            return true;
18        }
19    }
20    return false;
21 }
```

String、StringBuffer、StringBuilder 的区别？

可变性

String 是不可变的（后面会详细分析原因）。

StringBuilder 与 StringBuffer 都继承自 AbstractStringBuilder 类，在 AbstractStringBuilder 中也是使用字符数组保存字符串，不过没有使用 final 和 private 关键字修饰，最关键的是这个 AbstractStringBuilder 类还提供了很多修改字符串的方法比如 append 方法。

```
1  abstract class AbstractStringBuilder implements Appendable, CharSequence {
2      char[] value;
3      public AbstractStringBuilder append(String str) {
4          if (str == null)
5              return appendNull();
6          int len = str.length();
7          ensureCapacityInternal(count + len);
8          str.getChars(0, len, value, count);
9          count += len;
10         return this;
11     }
12     //...
13 }
```

线程安全性

String 中的对象是不可变的，也就可以理解为常量，线程安全。AbstractStringBuilder 是 StringBuilder 与 StringBuffer 的公共父类，定义了一些字符串的基本操作，如 expandCapacity、append、insert、indexOf 等公共方法。StringBuffer 对方法加了同步锁或者对调用的方法加了同步锁，所以是线程安全的。StringBuilder 并没有对方法进行加同步锁，所以是非线程安全的。

性能

每次对 String 类型进行改变的时候，都会生成一个新的 String 对象，然后将指针指向新的 String 对象。StringBuffer 每次都会对 StringBuffer 对象本身进行操作，而不是生成新的对象并改变对象引用。相同情况下使用 StringBuilder 相比使用 StringBuffer 仅能获得 10%~15% 左右的性能提升，但却要冒多线程不安全的风险。

对于三者使用的总结：

1. 操作少量的数据: 适用 String
2. 单线程操作字符串缓冲区下操作大量数据: 适用 StringBuilder
3. 多线程操作字符串缓冲区下操作大量数据: 适用 StringBuffer

String 为什么是不可变的？

String 类中使用 final 关键字修饰字符数组来保存字符串，所以 String 对象是不可变的。

```

1 public final class String implements java.io.Serializable,
   Comparable<String>, CharSequence {
2     private final char value[];
3     //...
4 }

```

🐞 修正：我们知道被 `final` 关键字修饰的类不能被继承，修饰的方法不能被重写，修饰的变量是基本数据类型则值不能改变，修饰的变量是引用类型则不能再指向其他对象。因此，`final` 关键字修饰的数组保存字符串并不是 `String` 不可变的根本原因，因为这个数组保存的字符串是可变的（`final` 修饰引用类型变量的情况）。

`String` 真正不可变有下面几点原因：

1. 保存字符串的数组被 `final` 修饰且为私有的，并且 `String` 类没有提供/暴露修改这个字符串的方法。
2. `String` 类被 `final` 修饰导致其不能被继承，进而避免了子类破坏 `String` 不可变。

相关阅读：[如何理解 String 类型值的不可变？ - 知乎提问open in new window](#)

补充（来自[issue 675open in new window](#)）：在 Java 9 之后，`String`、`StringBuilder` 与 `StringBuffer` 的实现改用 `byte` 数组存储字符串。

```

1 public final class String implements
   java.io.Serializable, Comparable<String>, CharSequence {
2     // @Stable 注解表示变量最多被修改一次，称为“稳定的”。
3     @Stable
4     private final byte[] value;
5 }
6
7 abstract class AbstractStringBuilder implements Appendable,
   CharSequence {
8     byte[] value;
9
10 }

```

异常

Exception 和 Error 有什么区别？

在 Java 中，所有的异常都有一个共同的祖先 `java.lang` 包中的 `Throwable` 类。`Throwable` 类有两个重要的子类：

- `Exception`：程序本身可以处理的异常，可以通过 `catch` 来进行捕获。`Exception` 又可以分为 Checked Exception (受检查异常，必须处理) 和 Unchecked Exception (不受检查异常，可以不处理)。
- `Error`：`Error` 属于程序无法处理的错误，我们没办法通过 `catch` 来进行捕获不建议通过 `catch` 捕获。例如 Java 虚拟机运行错误 (`Virtual MachineError`)、虚拟机内存不够错误 (`OutOfMemoryError`)、类定义错误 (`NoClassDefFoundError`) 等。这些异常发生时，Java 虚拟机 (JVM) 一般会选择线程终止。

Checked Exception 和 Unchecked Exception 有什么区别？

Checked Exception 即 受检查异常，Java 代码在编译过程中，如果受检查异常没有被 `catch` 或者 `throws` 关键字处理的话，就没办法通过编译。

比如下面这段 IO 操作的代码：

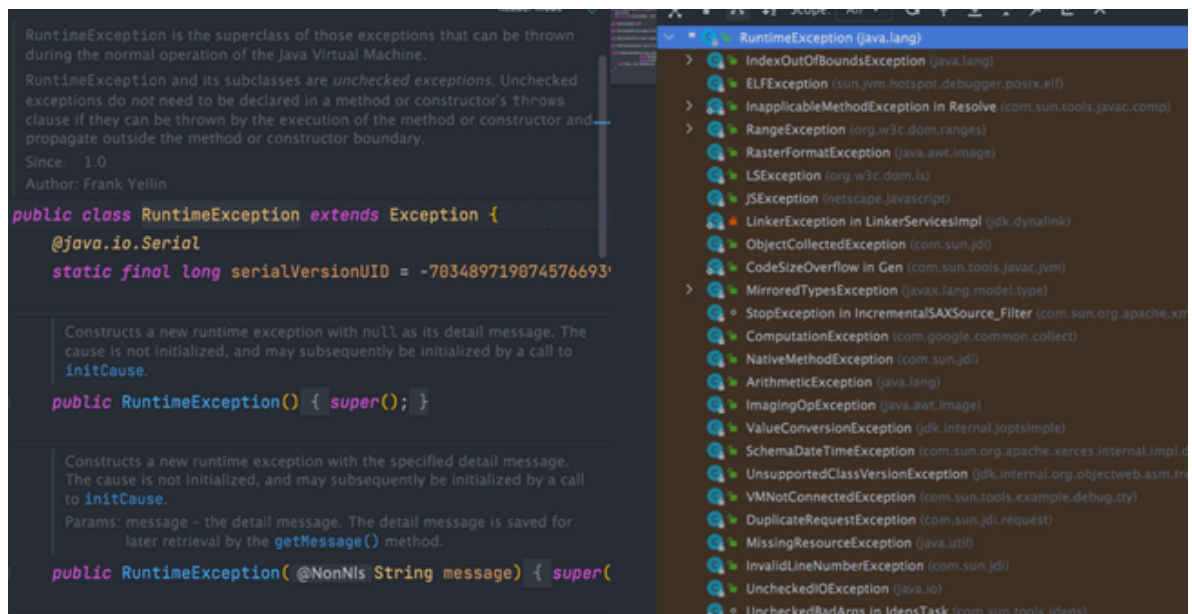


除了 `RuntimeException` 及其子类以外，其他的 `Exception` 类及其子类都属于受检查异常。常见的受检查异常有：IO 相关的异常、`ClassNotFoundException`、`SQLException`...

Unchecked Exception 即 不受检查异常，Java 代码在编译过程中，我们即使不处理不受检查异常也可以正常通过编译。

`RuntimeException` 及其子类都统称为非受检查异常，常见的有（建议记下来，日常开发中会经常用到）：

- `NullPointerException` (空指针错误)
- `IllegalArgumentException` (参数错误比如方法入参类型错误)
- `NumberFormatException` (字符串转换为数字格式错误，`IllegalArgumentException` 的子类)
- `ArrayIndexOutOfBoundsException` (数组越界错误)
- `ClassCastException` (类型转换错误)
- `ArithmeticException` (算术错误)
- `SecurityException` (安全错误比如权限不够)
- `UnsupportedOperationException` (不支持的操作错误比如重复创建同一用户)
-



Throwable 类常用方法有哪些？

- `String getMessage()` : 返回异常发生时的简要描述
- `String toString()` : 返回异常发生时的详细信息

- `String getLocalizedMessage()`: 返回异常对象的本地化信息。使用 `Throwable` 的子类覆盖这个方法, 可以生成本地化信息。如果子类没有覆盖该方法, 则该方法返回的信息与 `getMessage()` 返回的结果相同
- `void printStackTrace()`: 在控制台上打印 `Throwable` 对象封装的异常信息

try-catch-finally 如何使用?

- `try` 块: 用于捕获异常。其后可接零个或多个 `catch` 块, 如果没有 `catch` 块, 则必须跟一个 `finally` 块。
- `catch` 块: 用于处理 `try` 捕获到的异常。
- `finally` 块: 无论是否捕获或处理异常, `finally` 块里的语句都会被执行。当在 `try` 块或 `catch` 块中遇到 `return` 语句时, `finally` 语句块将在方法返回之前被执行。

代码示例:

```
1  try {
2      System.out.println("Try to do something");
3      throw new RuntimeException("RuntimeException");
4  } catch (Exception e) {
5      System.out.println("Catch Exception -> " + e.getMessage());
6  } finally {
7      System.out.println("Finally");
8  }
```

输出:

```
1  Try to do something
2  Catch Exception -> RuntimeException
3  Finally
```

注意: 不要在 finally 语句块中使用 return! 当 `try` 语句和 `finally` 语句中都有 `return` 语句时, `try` 语句块中的 `return` 语句会被忽略。这是因为 `try` 语句中的 `return` 返回值会先被暂存在一个本地变量中, 当执行到 `finally` 语句中的 `return` 之后, 这个本地变量的值就变为了 `finally` 语句中的 `return` 返回值。

[jvm 官方文档open in new window](#)中有明确提到:

If the `try` clause executes a *return*, the compiled code does the following:

1. Saves the return value (if any) in a local variable.
2. Executes a *jsr* to the code for the `finally` clause.
3. Upon return from the `finally` clause, returns the value saved in the local variable.

代码示例:

```

1 public static void main(String[] args) {
2     System.out.println(f(2));
3 }
4
5 public static int f(int value) {
6     try {
7         return value * value;
8     } finally {
9         if (value == 2) {
10             return 0;
11         }
12     }
13 }

```

输出:

```
1 | 0
```

finally 中的代码一定会执行吗?

不一定的! 在某些情况下, finally 中的代码不会被执行。

就比如说 finally 之前虚拟机被终止运行的话, finally 中的代码就不会被执行。

```

1 try {
2     System.out.println("Try to do something");
3     throw new RuntimeException("RuntimeException");
4 } catch (Exception e) {
5     System.out.println("Catch Exception -> " + e.getMessage());
6     // 终止当前正在运行的Java虚拟机
7     System.exit(1);
8 } finally {
9     System.out.println("Finally");
10 }

```

输出:

```

1 | Try to do something
2 | Catch Exception -> RuntimeException

```

另外, 在以下 2 种特殊情况下, finally 块的代码也不会被执行:

1. 程序所在的线程死亡。
2. 关闭 CPU。

相关 issue: <https://github.com/Snailclimb/JavaGuide/issues/190>open in new window。

 进阶一下: 从字节码角度分析 try catch finally 这个语法糖背后的实现原理。

如何使用 try-with-resources 代替 try-catch-finally?

1. **适用范围（资源的定义）**：任何实现 `java.lang.AutoCloseable` 或者 `java.io.Closeable` 的对象
2. **关闭资源和 finally 块的执行顺序**：在 `try-with-resources` 语句中，任何 catch 或 finally 块在声明的资源关闭后运行

《Effective Java》中明确指出：

面对必须要关闭的资源，我们总是应该优先使用 `try-with-resources` 而不是 `try-finally`。随之产生的代码更简短，更清晰，产生的异常对我们也更有用。`try-with-resources` 语句让我们更容易编写必须要关闭的资源的代码，若采用 `try-finally` 则几乎做不到这点。

Java 中类似于 `InputStream`、`OutputStream`、`Scanner`、`PrintWriter` 等的资源都需要我们调用 `close()` 方法来手动关闭，一般情况下我们都是通过 `try-catch-finally` 语句来实现这个需求，如下：

```
1 //读取文本文件的内容
2 Scanner scanner = null;
3 try {
4     scanner = new Scanner(new File("D://read.txt"));
5     while (scanner.hasNext()) {
6         System.out.println(scanner.nextLine());
7     }
8 } catch (FileNotFoundException e) {
9     e.printStackTrace();
10 } finally {
11     if (scanner != null) {
12         scanner.close();
13     }
14 }
```

使用 Java 7 之后的 `try-with-resources` 语句改造上面的代码：

```
1 try (Scanner scanner = new Scanner(new File("test.txt"))) {
2     while (scanner.hasNext()) {
3         System.out.println(scanner.nextLine());
4     }
5 } catch (FileNotFoundException fnfe) {
6     fnfe.printStackTrace();
7 }
```

当然多个资源需要关闭的时候，使用 `try-with-resources` 实现起来也非常简单，如果你还是用 `try-catch-finally` 可能会带来很多问题。

通过使用分号分隔，可以在 `try-with-resources` 块中声明多个资源。

```

1  try (BufferedInputStream bin = new BufferedInputStream(new
    FileInputStream(new File("test.txt")));
2      BufferedOutputStream bout = new BufferedOutputStream(new
    FileOutputStream(new File("out.txt")))) {
3      int b;
4      while ((b = bin.read()) != -1) {
5          bout.write(b);
6      }
7  }
8  catch (IOException e) {
9      e.printStackTrace();
10 }

```

异常使用有哪些需要注意的地方

- 不要把异常定义为静态变量，因为这样会导致异常栈信息错乱。每次手动抛出异常，我们都需要手动 new 一个异常对象抛出。
- 抛出的异常信息一定要有意义。
- 建议抛出更加具体的异常比如字符串转换为数字格式错误的时候应该抛出 `NumberFormatException` 而不是其父类 `IllegalArgumentException`。
- 使用日志打印异常之后就不要再抛出异常了（两者不要同时存在一段代码逻辑中）。

JAVAE

Spring

Springboot

操作系统

多线程

实现多线程有几种方法

1) 继承Thread类

(1) 定义Thread类的子类，并重写该类的run方法，该run方法的方法体就代表了线程要完成的任务。因此把run()方法称为执行体。

(2) 创建Thread子类的实例，即创建了线程对象。

(3) 调用线程对象的start()方法来启动该线程。

```

public class MyThread extends Thread {
    public void run() {
        System.out.println("MyThread.run()");
    }
}
MyThread myThread1 = new MyThread();
myThread1.start();

```

2) 实现Runnable接口

(1) 定义Runnable接口的实现类，并重写该接口的run()方法，该run()方法的方法体同样是该线程的线程执行体。

(2) 创建 Runnable实现类的实例，并依此实例作为Thread的target来创建Thread对象，该Thread对象才是真正的线程对象。

(3) 调用线程对象的start()方法来启动该线程

```
public class MyThread extends OtherClass implements Runnable {  
    public void run() {  
        System.out.println("MyThread.run()");  
    }  
}
```

3) 实现callable接口

1) 创建Callable接口的实现类，并实现call()方法，该call()方法将作为线程执行体，并且有返回值。

(2) 创建Callable实现类的实例，使用FutureTask类来包装Callable对象，该FutureTask对象封装了该Callable对象的call()方法的返回值。

(3) 使用FutureTask对象作为Thread对象的target创建并启动新线程。

(4) 调用FutureTask对象的get()方法来获得子线程执行结束后的返回值

```
public class SomeCallable extends OtherClass implements Callable {
```

```
1  @Override  
2  public V call() throws Exception {  
3      // TODO Auto-generated method stub  
4      return null;  
5  }
```

```
}
```

```
Callable oneCallable = new SomeCallable();
```

```
//由Callable创建一个FutureTask对象：
```

```
FutureTask oneTask = new FutureTask(oneCallable);
```

```
//注释：FutureTask是一个包装器，它通过接受Callable来创建，它同时实现了Future和Runnable接口。
```

```
//由FutureTask创建一个Thread对象：
```

```
Thread oneThread = new Thread(oneTask);
```

```
oneThread.start();
```

```
//至此，一个线程就创建完成了。
```

并发与并行的区别

- **并发**：两个及两个以上的作业在同一 **时间段** 内执行。
- **并行**：两个及两个以上的作业在同一 **时刻** 执行。

最关键的点是：是否是 **同时** 执行。

同步和异步的区别

- **同步**：发出一个调用之后，在没有得到结果之前，该调用就不可以返回，一直等待。
- **异步**：调用在发出之后，不用等待返回结果，该调用直接返回

sleep() 方法和 wait() 方法对比

共同点：两者都可以暂停线程的执行。

区别：

- `sleep()` 方法没有释放锁，而 `wait()` 方法释放了锁。
- `wait()` 通常被用于线程间交互/通信，`sleep()` 通常被用于暂停执行。
- `wait()` 方法被调用后，线程不会自动苏醒，需要别的线程调用同一个对象上的 `notify()` 或者 `notifyAll()` 方法。`sleep()` 方法执行完成后，线程会自动苏醒，或者也可以使用 `wait(long timeout)` 超时后线程会自动苏醒。
- `sleep()` 是 `Thread` 类的静态本地方法，`wait()` 则是 `Object` 类的本地方法。为什么这样设计呢？

可以直接调用 Thread 类的 run 方法吗？

这是另一个非常经典的 Java 多线程面试问题，而且在面试中会经常被问到。很简单，但是很多人都会答不上来！

new 一个 `Thread`，线程进入了新建状态。调用 `start()` 方法，会启动一个线程并使线程进入了就绪状态，当分配到时间片后就可以开始运行了。`start()` 会执行线程的相应准备工作，然后自动执行 `run()` 方法的内容，这是真正的多线程工作。但是，直接执行 `run()` 方法，会把 `run()` 方法当成一个 main 线程下的普通方法去执行，并不会在某个线程中执行它，所以这并不是多线程工作。

总结：调用 `start()` 方法方可启动线程并使线程进入就绪状态，直接执行 `run()` 方法的话不会以多线程的方式执行。

java内存管理

线程私有的：

- 程序计数器
- 虚拟机栈
- 本地方法栈

线程共享的：

- 堆
- 方法区
- 直接内存 (非运行时数据区的一部分)

栈内存

Java 虚拟机栈（后文简称栈）也是线程私有的，它的生命周期和线程相同，随着线程的创建而创建，随着线程的死亡而死亡。

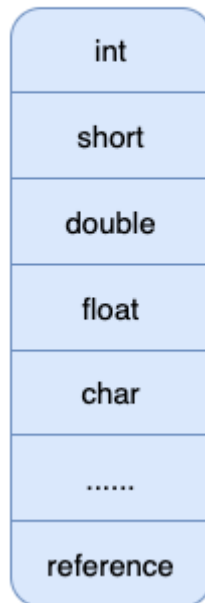
除了一些 Native 方法调用是通过本地方法栈实现的，其他所有的 Java 方法调用都是通过栈来实现的（也需要和其他运行时数据区域比如程序计数器配合）。

方法调用的数据需要通过栈进行传递，每一次方法调用都会有一个对应的栈帧被压入栈中，每一个方法调用结束后，都会有一个栈帧被弹出。

栈由一个个栈帧组成，而每个栈帧中都拥有：局部变量表、操作数栈、动态链接、方法返回地址。和数据结构上的栈类似，两者都是先进后出的数据结构，只支持出栈和入栈两种操作。

局部变量表 主要存放了编译期可知的各种数据类型（boolean、byte、char、short、int、float、long、double）、对象引用（reference 类型，它不同于对象本身，可能是一个指向对象起始地址的引用指针，也可能是指向一个代表对象的句柄或其他与此对象相关的位置）。

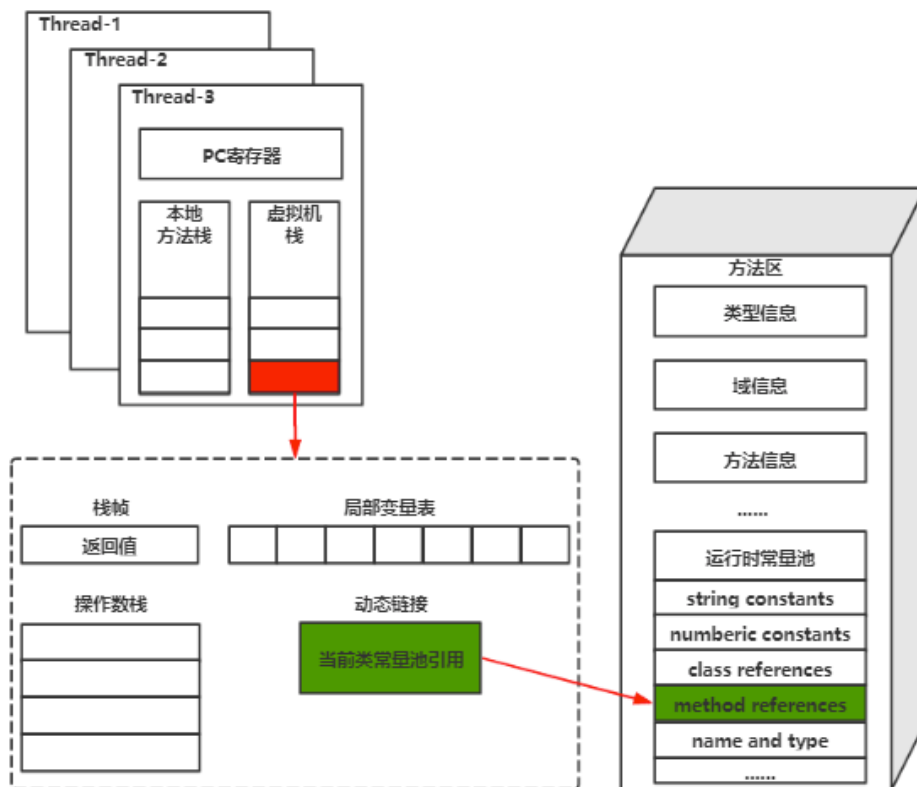
Java栈 局部变量表



操作数栈 主要作为方法调用的中转站使用，用于存放方法执行过程中产生的中间计算结果。另外，计算过程中产生的临时变量也会放在操作数栈中。

动态链接 主要服务一个方法需要调用其他方法的场景。在 Java 源文件被编译成字节码文件时，所有的变量和方法引用都作为符号引用（Symbolic Reference）保存在 Class 文件的常量池里。当一个方法要调用其他方法，需要将常量池中指向方法的符号引用转化为其在内存地址中的直接引用。动态链接的作用就是为了将符号引用转换为调用方法的直接引用。

动态链接（或指向运行时常量池的方法引用）：



栈空间虽然不是无限的，但一般正常调用的情况下是不会出现问题的。不过，如果函数调用陷入无限循环的话，就会导致栈中被压入太多栈帧而占用太多空间，导致栈空间过深。那么当线程请求栈的深度超过当前 Java 虚拟机栈的最大深度的时候，就抛出 `StackOverflowError` 错误。

Java 方法有两种返回方式，一种是 `return` 语句正常返回，一种是抛出异常。不管哪种返回方式，都会导致栈帧被弹出。也就是说，**栈帧随着方法调用而创建，随着方法结束而销毁。无论方法正常完成还是异常完成都算作方法结束。**

堆内存

Java 虚拟机所管理的内存中最大的一块，Java 堆是所有线程共享的一块内存区域，在虚拟机启动时创建。**此内存区域的唯一目的就是存放对象实例，几乎所有的对象实例以及数组都在这里分配内存。**

Java 世界中“几乎”所有的对象都在堆中分配，但是，随着 JIT 编译器的发展与逃逸分析技术逐渐成熟，栈上分配、标量替换优化技术将会导致一些微妙的变化，所有的对象都分配到堆上也渐渐变得不那么“绝对”了。从 JDK 1.7 开始已经默认开启逃逸分析，如果某些方法中的对象引用没有被返回或者未被外面使用（也就是未逃逸出去），那么对象可以直接在栈上分配内存。

Java 堆是垃圾收集器管理的主要区域，因此也被称作 **GC 堆（Garbage Collected Heap）**。从垃圾回收的角度，由于现在收集器基本都采用分代垃圾收集算法，所以 Java 堆还可以细分为：新生代和老年代；再细致一点有：Eden、Survivor、Old 等空间。

方法区

方法区属于是 JVM 运行时数据区域的一块逻辑区域，是各个线程共享的内存区域。

当虚拟机要使用一个类时，它需要读取并解析 Class 文件获取相关信息，再将信息存入到方法区。方法区会存储已被虚拟机加载的 **类信息、字段信息、方法信息、常量、静态变量、即时编译器编译后的代码缓存等数据。**

java垃圾回收机制

主要针对堆内存进行内存管理

使用多线程可能带来什么问题？

并发编程的目的就是为了能提高程序的执行效率提高程序运行速度，但是并发编程并不总是能提高程序运行速度的，而且并发编程可能会遇到很多问题，比如：内存泄漏、死锁、线程不安全等等。

如何预防和避免线程死锁？

如何预防死锁？ 破坏死锁的产生的必要条件即可：

1. **破坏请求与保持条件**：一次性申请所有的资源。
2. **破坏不剥夺条件**：占用部分资源的线程进一步申请其他资源时，如果申请不到，可以主动释放它占有的资源。
3. **破坏循环等待条件**：靠按序申请资源来预防。按某一顺序申请资源，释放资源则反序释放。破坏循环等待条件。

如何避免死锁？

避免死锁就是在资源分配时，借助于算法（比如银行家算法）对资源分配进行计算评估，使其进入安全状态。

安全状态 指的是系统能够按照某种线程推进顺序（P1、P2、P3.....Pn）来为每个线程分配所需资源，直到满足每个线程对资源的最大需求，使每个线程都可顺利完成。称 `<P1、P2、P3.....Pn>` 序列为安全序列。

我们对线程 2 的代码修改成下面这样就不会产生死锁了。

```

1  new Thread() -> {
2      synchronized (resource1) {
3          System.out.println(Thread.currentThread() + "get
resource1");
4          try {
5              Thread.sleep(1000);
6          } catch (InterruptedException e) {
7              e.printStackTrace();
8          }
9          System.out.println(Thread.currentThread() + "waiting get
resource2");
10         synchronized (resource2) {
11             System.out.println(Thread.currentThread() + "get
resource2");
12         }
13     }
14     }, "线程 2").start();

```

输出:

```

1  Thread[线程 1,5,main]get resource1
2  Thread[线程 1,5,main]waiting get resource2
3  Thread[线程 1,5,main]get resource2
4  Thread[线程 2,5,main]get resource1
5  Thread[线程 2,5,main]waiting get resource2
6  Thread[线程 2,5,main]get resource2
7
8  Process finished with exit code 0

```

我们分析一下上面的代码为什么避免了死锁的发生?

线程 1 首先获得到 resource1 的监视器锁,这时候线程 2 就获取不到了。然后线程 1 再去获取 resource2 的监视器锁, 可以获取到。然后线程 1 释放了对 resource1、resource2 的监视器锁的占用, 线程 2 获取到就可以执行了。这样就破坏了破坏循环等待条件, 因此避免了死锁。

线程上下文切换

多线程的上下文切换是指 CPU 控制权由一个已经正在运行的线程切换到另外一个就绪并等待获取 CPU 执行权的线程的过程。

可能有以下原因:

线程的 cpu 时间片用完(每个线程轮流执行, 看前面并发的概念)

垃圾回收

有更高优先级的线程需要运行

线程自己调用了 sleep、yield、wait、join、park、synchronized、lock 等方法

当 Context Switch 发生时, 需要由操作系统保存当前线程的状态, 并恢复另一个线程的状态

框架

spring框架的初步理解

IOC和AOP

IoC:

Inversion of Control 控制反转。我们以前开发，在一个类中使用其他类对象的时候都是采用new的方式直接获取，或者高级一点是通过反射的方式得到需要的对象实例。这就造成了程序的耦合度非常高，一个类的运行，严重依赖于其他的类。并且还会出现程序中硬编码的情况。而spring中的IoC很好的解决了该问题，我们在一个类中使用其他类对象时，只需要定义一个接口类型的类成员变量，由使用者在使用时为我们注入具体的实现类对象，从而降低了程序的耦合度。

它是spring的核心之一。或者说是spring的基础核心，spring的其余核心功能都必须有IoC的支持。控制反转指的是，我们在获取对象的时候，由之前的主动出击，变成了被动接收。也就是说，在编写某个类时，只需要提供一个接口类型的类成员，并不需要关系具体的实现类，而是由使用者在使用时提供。这就降低了类和类之间的耦合度。

实现IoC的思想就只有两种：依赖注入（Dependency Injection，简称DI）和依赖查找（Dependency Lookup）。

而依赖注入使用的更广泛一些。例如：构造函数注入,set方法注入等等。

首先，明确依赖注入是实现控制反转的一种思想（另一种是依赖查找）。

其次，在开发过程中，我们需要某个类的实例时，是由使用者为我们提供该类的实例。而不是自己去获取。

最后，实现依赖注入的方式可以是使用构造方法注入或者set方法注入两种方式。

在spring中注入的方式就有很多了，比如constructor-arg元素，property元素，p名称空间等等。

在持久层、业务层和控制层分别采用 @Repository、@Service 和 @Controller 对分层中的类进行注释，而用 @Component 对那些比较中立的类进行注释

AOP:

Aspect Oriented Programming 面向切面编程。

我们之前学习的java语言，号称是面向对象编程，它有自己的优势。但是也存在着一些弊端。举例说明，在实际开发中，我们都会有一根业务主线，即客户（甲方）的需求。而程序员要做的就是围绕业务主线上的需求，实现功能（实现功能的方法我们叫做业务核心方法）。但是不可避免的，会有一些功能与业务主线没有关系，却又不能不做，比如权限的控制，事务的控制，日志的记录等等，这些功能绝大多数时候和业务主线没有关系，但是却和很多业务核心方法交织在一起，使我们的开发变得麻烦，并且冗余代码增多。而spring的提供了一种思想，把这些和业务主线没有关系的功能剥离出来，而在需要使用这些公共方法时，适时适地的把它加到我们的代码中去，使程序员在开发时，把更多的精力放在理解需求，实现业务核心功能上，并且让我们的代码变得简洁。这种思想就是面向切面编程。

什么是 Spring Bean?

简单来说，Bean 代指的就是那些被 IoC 容器所管理的对象。

我们需要告诉 IoC 容器帮助我们管理哪些对象，这个是通过配置元数据来定义的。配置元数据可以是 XML 文件、注解或者 Java 配置类。

将一个类声明为 Bean 的注解有哪些?

- **@Component** : 通用的注解，可标注任意类为 Spring 组件。如果一个 Bean 不知道属于哪个层，可以使用 @Component 注解标注。
- **@Repository** : 对应持久层即 Dao 层，主要用于数据库相关操作。
- **@Service** : 对应服务层，主要涉及一些复杂的逻辑，需要用到 Dao 层。

- `@Controller` : 对应 Spring MVC 控制层，主要用户接受用户请求并调用 Service 层返回数据给前端页面。

#@Component 和 @Bean 的区别是什么？

- `@Component` 注解作用于类，而 `@Bean` 注解作用于方法。
- `@Component` 通常是通过类路径扫描来自动侦测以及自动装配到 Spring 容器中（我们可以使用 `@ComponentScan` 注解定义要扫描的路径从中找出标识了需要装配的类自动装配到 Spring 的 bean 容器中）。`@Bean` 注解通常是在标有该注解的方法中定义产生这个 bean, `@Bean` 告诉了 Spring 这是某个类的实例，当我需要它的时候还给我。
- `@Bean` 注解比 `@Component` 注解的自定义性更强，而且很多地方我们只能通过 `@Bean` 注解来注册 bean。比如当我们引用第三方库中的类需要装配到 Spring 容器时，则只能通过 `@Bean` 来实现。

@Bean 注解使用示例：

```
1  @Configuration
2  public class AppConfig {
3      @Bean
4      public TransferService transferService() {
5          return new TransferServiceImpl();
6      }
7  }
8  }
```

上面的代码相当于下面的 xml 配置

```
1  <beans>
2      <bean id="transferService" class="com.acme.TransferServiceImpl"/>
3  </beans>
```

下面这个例子是通过 `@Component` 无法实现的。

```
1  @Bean
2  public OneService getService(status) {
3      case (status) {
4          when 1:
5              return new serviceImpl1();
6          when 2:
7              return new serviceImpl2();
8          when 3:
9              return new serviceImpl3();
10     }
11 }
```

注入-bean-的注解有哪些)注入 Bean 的注解有哪些？

Spring 内置的 `@Autowired` 以及 JDK 内置的 `@Resource` 和 `@Inject` 都可以用于注入 Bean。

Annotaion	Package	Source
<code>@Autowired</code>	<code>org.springframework.bean.factory</code>	Spring 2.5+
<code>@Resource</code>	<code>javax.annotation</code>	Java JSR-250
<code>@Inject</code>	<code>javax.inject</code>	Java JSR-330

`@Autowired` 和 `@Resource` 使用的比较多一些。

@Autowired 和 @Resource 的区别是什么？

`Autowired` 属于 Spring 内置的注解，默认的注入方式为 `byType`（根据类型进行匹配），也就是说会优先根据接口类型去匹配并注入 Bean（接口的实现类）。

这会有什么问题呢？ 当一个接口存在多个实现类的话，`byType` 这种方式就无法正确注入对象了，因为这个时候 Spring 会同时找到多个满足条件的选择，默认情况下它自己不知道选择哪一个。

这种情况下，注入方式会变为 `byName`（根据名称进行匹配），这个名称通常就是类名（首字母小写）。就比如说下面代码中的 `smsService` 就是我这里所说的名称，这样应该比较好理解了吧。

```
1 // smsService 就是我们上面所说的名称
2 @Autowired
3 private SmsService smsService;
```

举个例子，`SmsService` 接口有两个实现类：`SmsServiceImpl1` 和 `SmsServiceImpl2`，且它们都被 Spring 容器所管理。

```
1 // 报错，byName 和 byType 都无法匹配到 bean
2 @Autowired
3 private SmsService smsService;
4 // 正确注入 SmsServiceImpl1 对象对应的 bean
5 @Autowired
6 private SmsService smsServiceImpl1;
7 // 正确注入 SmsServiceImpl1 对象对应的 bean
8 // smsServiceImpl1 就是我们上面所说的名称
9 @Autowired
10 @Qualifier(value = "smsServiceImpl1")
11 private SmsService smsService;
```

我们还是建议通过 `@Qualifier` 注解来显示指定名称而不是依赖变量的名称。

`@Resource` 属于 JDK 提供的注解，默认注入方式为 `byName`。如果无法通过名称匹配到对应的 Bean 的话，注入方式会变为 `byType`。

`@Resource` 有两个比较重要且日常开发常用的属性：`name`（名称）、`type`（类型）。

```
1 public @interface Resource {
2     String name() default "";
3     Class<?> type() default Object.class;
4 }
```

如果仅指定 `name` 属性则注入方式为 `byName`，如果仅指定 `type` 属性则注入方式为 `byType`，如果同时指定 `name` 和 `type` 属性（不建议这么做）则注入方式为 `byType + byName`。

```
1 // 报错, byName 和 byType 都无法匹配到 bean
2 @Resource
3 private SmsService smsService;
4 // 正确注入 SmsServiceImpl 对象对应的 bean
5 @Resource
6 private SmsService smsServiceImpl;
7 // 正确注入 SmsServiceImpl 对象对应的 bean（比较推荐这种方式）
8 @Resource(name = "smsServiceImpl")
9 private SmsService smsService;
```

简单总结一下：

- `@Autowired` 是 Spring 提供的注解，`@Resource` 是 JDK 提供的注解。
- `Autowired` 默认的注入方式为 `byType`（根据类型进行匹配），`@Resource` 默认注入方式为 `byName`（根据名称进行匹配）。
- 当一个接口存在多个实现类的情况下，`@Autowired` 和 `@Resource` 都需要通过名称才能正确匹配到对应的 Bean。`Autowired` 可以通过 `@Qualifier` 注解来显示指定名称，`@Resource` 可以通过 `name` 属性来显示指定名称。

#Bean 的作用域有哪些？

Spring 中 Bean 的作用域通常有下面几种：

- **singleton** : IoC 容器中只有唯一的 bean 实例。Spring 中的 bean 默认都是单例的，是对单例设计模式的应用。
- **prototype** : 每次获取都会创建一个新的 bean 实例。也就是说，连续 `getBean()` 两次，得到的是不同的 Bean 实例。
- **request**（仅 Web 应用可用）：每一次 HTTP 请求都会产生一个新的 bean（请求 bean），该 bean 仅在当前 HTTP request 内有效。
- **session**（仅 Web 应用可用）：每一次来自新 session 的 HTTP 请求都会产生一个新的 bean（会话 bean），该 bean 仅在当前 HTTP session 内有效。
- **application/global-session**（仅 Web 应用可用）：每个 Web 应用在启动时创建一个 Bean（应用 Bean），该 bean 仅在当前应用启动时间内有效。
- **websocket**（仅 Web 应用可用）：每一次 WebSocket 会话产生一个新的 bean。

如何配置 bean 的作用域呢？

xml 方式：

```
1 <bean id="..." class="..." scope="singleton"></bean>
```

注解方式:

```
1 @Bean
2 @Scope(value = ConfigurableBeanFactory.SCOPE_PROTOTYPE)
3 public Person personPrototype() {
4     return new Person();
5 }
```

Spring MVC 的核心组件有哪些?

记住了下面这些组件,也就记住了 SpringMVC 的工作原理。

- **DispatcherServlet**: **核心的中央处理器**,负责接收请求、分发,并给予客户端响应。
- **HandlerMapping**: **处理器映射器**,根据 uri 去匹配查找能处理的 **Handler**, 并将请求涉及到的拦截器和 **Handler** 一起封装。
- **HandlerAdapter**: **处理器适配器**,根据 **HandlerMapping** 找到的 **Handler**, 适配执行对应的 **Handler**;
- **Handler**: **请求处理器**,处理实际请求的处理器。
- **ViewResolver**: **视图解析器**,根据 **Handler** 返回的逻辑视图 / 视图,解析并渲染真正的视图,并传递给 **DispatcherServlet** 响应客户端

Mybatis框架的初步理解

SqlSessionFactory对象 创建SqlSession的工厂对象 连接数据库

SqlSession对象 以mybatis的方式操作数据库-内部整合JDBC连接

编程步骤

- 1.创建SqlSessionFactory
- 2.通过SqlSessionFactory创建SqlSession
- 3.通过sqlSession执行数据库操作
- 4.调用session.commit()提交事物
- 5.调用session.close()关闭会话

mybatis的一级缓存和二级缓存:

Mybatis首先缓存中查询结果集,如果没有则查数据库,如果有则从缓存中取出返回结果集就不走数据库.Mybatis内部缓存使用HashMap, key为HashCode+sqlId+Sql语句.Value为从查询出来映射生成的Java对象Mybatis的二级缓存即查询缓存,它的作用域是一个mapper的namespace,记在同一个namespace中查询sql可以缓存中获取数据.二级缓存可以跨SqlSession的

算法

排序算法稳定性

只有冒泡、归并、插入排序是稳定的,其他都不稳定